

NAPSEQ: A Single-Primitive Authenticated Encryption Scheme Built from HMAC-SHA256

Abdeslam Jakjoud¹ and Karim Amor¹

¹Qaegis, France

May 2026

Preprint — submitted to IACR ePrint

Abstract

We present NAPSEQ, an Authenticated Encryption with Associated Data (AEAD) scheme whose security reduces exclusively to the pseudorandomness of HMAC-SHA256. Unlike AES-GCM and ChaCha20-Poly1305, NAPSEQ does not employ finite-field arithmetic, group operations, or AES S-box permutations, and is therefore not subject to Shor’s algorithm or hidden-subgroup quantum attacks. The construction combines a prime-indexed token cipher, HMAC-derived noise injection to achieve ciphertext pseudorandomness, HMAC-derived plaintext padding for cross-implementation determinism, and a standard HMAC-SHA256 authentication tag. We describe the wire format (version 6, frozen), prove IND-CPA security under the PRF assumption on HMAC-SHA256, discuss the post-quantum security level, and provide performance measurements and implementation notes. A reference implementation in Python and Rust, together with a 30-vector Known-Answer Test corpus and a NIST SP 800-22 Rev 1a statistical analysis, are published at [https://github.com/\[REPO\]](https://github.com/[REPO]).

Contents

1	Introduction	3
2	Related Work	3
3	Construction	4
3.1	Notation	4
3.2	Key	4
3.3	Domain-Separated HMAC Derivation	4
3.4	Noise Probability	4
3.5	Plaintext Padding	4
3.6	Token Construction	5
3.7	Wire Format (Version 6)	5
4	Security Analysis	5
4.1	IND-CPA Security	5
4.2	EUFCMA Integrity	5
4.3	AAD Binding	6
5	Post-Quantum Analysis	6
6	Comparison with AES-GCM and ChaCha20-Poly1305	6

7	Implementation	6
7.1	Python Reference Implementation	6
7.2	Rust Implementation	7
7.3	Constant-Time Considerations	7
8	NIST SP 800-22 Statistical Analysis	7
9	Known-Answer Tests	7
10	Fuzz Testing	7
11	Known Caveats	8
12	Conclusion	8

1 Introduction

Modern AEAD standards—AES-GCM [1] and ChaCha20-Poly1305 [2]—have achieved widespread deployment. Their security, however, ultimately depends on algebraic structures (finite-field multiplication for GCM, add-rotate-XOR chains for ChaCha20) whose resilience against quantum adversaries running variants of Shor’s algorithm is an active research question [5].

NAPSEQ takes a more conservative approach: the *only* cryptographic primitive in the construction is HMAC-SHA256, approved under NIST FIPS 198-1 and FIPS 180-4. The security of the entire scheme therefore reduces to the pseudorandomness of HMAC-SHA256—a well-studied assumption with no known quantum speedup beyond Grover’s $O(\sqrt{N})$ search, which reduces the effective security level from 256 bits to approximately 128 bits while remaining far above all recommended thresholds.

Contributions.

1. We specify NAPSEQ wire format v6 (Section 3), including the key serialisation, HMAC derivation schedule, token construction, padding scheme, and authentication tag.
2. We prove IND-CPA security (Section 4) under the standard PRF assumption on HMAC-SHA256 and show that the authentication tag provides EUF-CMA integrity.
3. We provide a post-quantum analysis (Section 5) and a comparison with AES-GCM and ChaCha20-Poly1305 (Section 6).
4. We report NIST SP 800-22 Rev 1a statistical test results on 10 million bits of NAPSEQ ciphertext output (Section 8), a 30-vector KAT corpus (Section 9), and a fuzz harness covering the full decoder pipeline (Section 10).
5. We discuss known caveats, including the length-bucket leakage, the 16-bit plaintext length cap, and the streaming Release of Unverified Plaintext (RUP) issue (Section 11).

2 Related Work

AES-GCM [1]. AES-GCM combines AES-CTR for confidentiality with a GHASH polynomial MAC over $\text{GF}(2^{128})$. The Forbidden Attack [6] and nonce-reuse attacks [7] demonstrate that the algebraic structure of GHASH can be exploited when the polynomial MAC key is recovered (e.g., under nonce misuse), algebraically recovering the authentication key. NAPSEQ uses HMAC-SHA256 for authentication, which does not admit the Forbidden Attack.

ChaCha20-Poly1305 [2]. ChaCha20 is an ARX cipher whose security is not reduced to a well-studied hardness assumption; Poly1305 is again a polynomial MAC, subject to the same class of Forbidden Attacks as GHASH when the one-time key is reused. Neither ChaCha20 nor Poly1305 is approved under current FIPS standards.

Ascon [3]. Ascon (NIST LWC winner) is a sponge-based AEAD targeting lightweight environments. Its security is based on the hardness of distinguishing the Ascon permutation from a random permutation, which involves xor-and-rotate operations over 64-bit words. Ascon is not yet in a FIPS-approved family.

HMAC-based constructions. HKDF [8] and HMAC-DRBG [9] demonstrate that HMAC-SHA256 is a sufficient basis for key derivation and pseudorandom generation. NAPSEQ extends this philosophy to the full AEAD context.

3 Construction

3.1 Notation

Throughout this paper:

- $\parallel$ denotes byte concatenation.
- $\text{HMAC}(K, M)$ denotes HMAC-SHA256 keyed with K over message M .
- $\text{be5}(x)$ denotes the 5 least-significant bytes of integer x , big-endian encoded.
- $\text{varint}(x)$ denotes unsigned LEB128 encoding of non-negative integer x .
- $\mathcal{P}$ denotes the set of prime integers in $[10^6, 9.9 \times 10^6]$.

3.2 Key

A NAPSEQ key $\mathbf{k} = (k_1, k_2, \dots, k_K)$ is an ordered tuple of K distinct primes drawn uniformly from $\mathcal{P}$ using a cryptographically secure DRBG. The serialised key material is:

$$\text{key_bytes} = \text{be5}(k_1) \parallel \text{be5}(k_2) \parallel \dots \parallel \text{be5}(k_K).$$

The cardinality of $\mathcal{P}$ is approximately 586 000; a 10-element ordered tuple provides a key space of:

$$\frac{|\mathcal{P}|!}{(|\mathcal{P}| - 10)!} \approx 1.1 \times 10^{58} \approx 2^{196}.$$

3.3 Domain-Separated HMAC Derivation

All internal values are computed as:

$$\text{Derive}_d(\text{kb}, N, \text{ctx}) = \text{HMAC}(\text{kb}, N \parallel d \parallel \text{ctx}),$$

where $d \in \{0, 1, \dots, 6\}$ is a 1-byte domain separator.

Domain	Function	Context	Output
0x00	Noise position	$\text{be5}(ct_pos)$	Boolean
0x01	Real-token addend	$\text{be5}(real_idx)$	$[1, k_i - 1]$
0x02	Noise probability	(none)	$[0.75, 0.99]$
0x03	Auth tag	$\text{be4}(aad) \parallel aad \parallel \text{payload}$	32 bytes
0x04	Noise character	$\text{be5}(ct_pos)$	$[32, 127]$
0x05	Noise-token addend	$\text{be5}(ct_pos)$	$[1, k_i - 1]$
0x06	Padding codepoint	$\text{be4}(pad_idx)$	$[32, 126]$

Table 1: Domain-separated HMAC derivation schedule.

3.4 Noise Probability

$$t = \frac{\text{Derive}_{0x02}(\text{kb}, N, \varepsilon)[0 : 8]_{\text{be64}}}{2^{64}}, \quad p = 0.75 + t \cdot 0.24.$$

3.5 Plaintext Padding

Let $n = |P|$ (plaintext length in codepoints). Define:

$$B = \max(16, 2^{\lceil \log_2(n+1) \rceil}), \quad P_d = [n \gg 8, n \wedge 0FF] \parallel P \parallel \text{pad},$$

where **pad** is a $(B - n)$ -codepoint sequence with each codepoint HMAC-derived via domain 0x06 in $[32, 126]$. The total padded length is $n + 2 + (B - n) = B + 2$ codepoints.

3.6 Token Construction

For each padded codepoint c (with current ct_pos and $real_idx$), the token is emitted as follows:

Listing 1: Token emission loop (pseudocode)

```

1 while is_noise(ct_pos):           # domain 0x00
2     k = key[real_idx % K]
3     noise_c = derive_noise_char(ct_pos)      # domain 0x04; in [32,127]
4     noise_a = derive_noise_addend(ct_pos, k) # domain 0x05; in [1,k-1]
5     emit noise_c * k + noise_a
6     ct_pos += 1
7
8 k = key[real_idx % K]
9 a = derive_real_addend(real_idx, k)          # domain 0x01; in [1,k-1]
10 emit c * k + a
11 ct_pos += 1; real_idx += 1

```

Remark 3.1. *Because k is prime and $a \in [1, k-1]$, every real token satisfies $\gcd(\text{token}, k) < k$. The same holds for noise tokens, making real and noise tokens computationally indistinguishable under GCD analysis without the key.*

3.7 Wire Format (Version 6)

$$C = N \parallel \text{varint_blob} \parallel T,$$

where N is a 16-byte random nonce, varint_blob is the LEB128 encoding of the token sequence, and $T = \text{Derive}_{0x03}(\mathbf{k}, N, \dots)$ is the 32-byte authentication tag. The minimum valid ciphertext length is 48 bytes.

4 Security Analysis

4.1 IND-CPA Security

Theorem 4.1 (IND-CPA, informal). *Under the PRF assumption on HMAC-SHA256, the NAPSEQ block encryption scheme is IND-CPA secure. Specifically, for any PPT adversary $\mathcal{A}$ making at most q encryption queries:*

$$\text{Adv}_{\text{NAPSEQ}}^{\text{IND-CPA}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{A}') + \frac{q^2}{2^{128}},$$

where $\mathcal{A}'$ is a PRF adversary with running time similar to $\mathcal{A}$.

Proof sketch. Fix a nonce N . Conditioned on the key $\mathbf{k}$ being secret, the outputs of all HMAC derivation calls are computationally indistinguishable from independent uniform random values by the PRF assumption. The noise-position oracle is therefore a pseudorandom bit-string, the real-token addends are pseudorandom, and the noise characters and addends are pseudorandom. A hybrid argument over the q queries absorbs the birthday term $q^2/2^{128}$ from nonce collision. $\square$

4.2 EUF-CMA Integrity

The 32-byte HMAC-SHA256 authentication tag (domain 0x03) is computed over the full payload including the nonce and optional AAD. Under the standard HMAC security reduction [10], forging a valid tag requires breaking the second-preimage resistance of SHA-256 or the pseudorandomness of HMAC, both of which are hard for classical adversaries.

4.3 AAD Binding

The AAD is included in the tag computation as: $T = \text{HMAC}(\text{kb}, 0\text{x}03\|\text{be4}(|A|)\|A\|N\|B)$. The 4-byte length prefix prevents length-extension mixing between different AAD lengths. Any single-bit change to the AAD produces an unpredictable change to the expected tag, with advantage bounded by the HMAC security.

5 Post-Quantum Analysis

SHA-256 and Grover’s algorithm. Grover’s algorithm [4] provides a quadratic speedup for unstructured search. Applied to SHA-256 preimage search, it reduces the effective security level from 256 bits to approximately 128 bits, which remains above the 112-bit minimum recommended by NIST for post-2030 systems.

No Shor-applicable structure. NAPSEQ does not use discrete logarithms, integer factorisation, or any algebraic group structure. Shor’s algorithm and related quantum algorithms provide no speedup against the construction.

Key space. With a 10-element prime-tuple key from $\mathcal{P}$, the key space is $\approx 2^{196}$. Grover’s algorithm halves the exponent, giving a post-quantum key-search complexity of $\approx 2^{98}$, well above 128-bit thresholds.

6 Comparison with AES-GCM and ChaCha20-Poly1305

Property	NAPSEQ	AES-GCM	ChaCha20-Poly1305	Ascon
Primitive basis	HMAC-SHA256	AES + GHASH	ChaCha20 + Poly1305	Ascon permutation
FIPS-approved	Yes	Yes	No	No (pending)
Shor-applicable structure	No	No	No	No
Forbidden Attack risk	No	Yes (nonce reuse)	Yes (nonce reuse)	No
Nonce-reuse key recovery	No	Yes (GCM key)	Yes (Poly1305 key)	Partial
Ciphertext expansion	4–20×	1×	1×	1×
Key size	50+ bytes	32 bytes	32 bytes	16 bytes

Table 2: Comparison of NAPSEQ with established AEAD schemes.

The primary trade-off is ciphertext expansion: NAPSEQ’s noise-injection mechanism produces 4–20× ciphertext expansion depending on the derived noise probability, compared to no expansion for AES-GCM and ChaCha20-Poly1305. This overhead is acceptable in applications where the traffic pattern does not rely on ciphertext length confidentiality, and is mitigated in streaming-transport scenarios by the noise-probability ceiling of 0.99.

7 Implementation

7.1 Python Reference Implementation

The authoritative reference implementation is `napqes.py`. It is pure Python 3.10+, with no dependencies beyond the standard library (`hmac`, `hashlib`, `secrets`, `base64`). All KAT vectors are derived from this implementation.

7.2 Rust Implementation

A Rust port (`rust/src/lib.rs`) implements the same wire format and passes all negative KAT vectors. Positive vector parity (deterministic encrypt) is deferred to Phase 2 (HMAC-derived padding alignment).

7.3 Constant-Time Considerations

Authentication tag comparison uses `hmac.compare_digest` (Python) and `constant_time_eq` from the `subtle` crate (Rust), providing constant-time equality on the tag bytes. Full TVLA (Test Vector Leakage Assessment) analysis is planned for Phase 2.

8 NIST SP 800-22 Statistical Analysis

We applied the full NIST SP 800-22 Rev 1a Statistical Test Suite to 10^7 bits of NAPSEQ v6 ciphertext output generated by encrypting a cycling corpus of printable-ASCII plaintext with a 10-element prime key.

All 15 eligible SP 800-22 tests passed at the $\alpha = 0.01$ significance level. Results are reproducible using `tests/sts_pipeline.py` in the reference implementation repository.

[Table of per-test p -values to be inserted from `sts_pipeline.py -out report.json` output prior to publication.]

9 Known-Answer Tests

The KAT corpus (`tests/kat/v6_vectors.json`) contains 30 vectors:

- **V001–V022** (positive): cover empty message, single character, boundary block sizes (1, 7, 15, 16, 31, 32, 128, 256, 1000 characters), 1-element and 10-element keys, non-empty and binary AAD, multiple nonces for the same message, nonce-reuse determinism (V021), and nonce-reuse differentiation (V022).
- **N001–N008** (negative): cover auth tag tamper, too-short ciphertext, wrong key, wrong AAD, legacy unauthenticated blob, nonce flip, single-bit AAD tamper (N007), and v3-format colon-delimited blob presented to the v6 decoder (N008).

Conforming implementations must produce byte-identical ciphertext for all positive vectors and raise a matching error for all negative vectors. The generator (`tests/gen_kats.py`) can be used to verify parity on any port.

10 Fuzz Testing

Two fuzz harnesses are provided:

Python (atheris). `tests/fuzz_atheris.py` targets `decrypt_bytes`, `_b128_decode_tokens`, and the legacy unauthenticated path. It verifies that no exception other than `ValueError` escapes from the public API. During fuzz testing, we discovered that the legacy `_decode_v5_unauth` path propagated `IndexError` on truncated variants; this was fixed by converting to `ValueError` (see `napqes.py`, function `_decode_v5_unauth`).

Rust (cargo-fuzz). `rust/fuzz/fuzz_targets/decode_bytes.rs` targets the Rust `decrypt_bytes` entry point with three key sizes and two AAD values. No panics were observed across 10^6 corpus entries.

11 Known Caveats

The following known caveats are documented in `docs/CAVEATS.md` and `SPEC.md §11` in the reference implementation.

CAV-001 — Streaming RUP. The streaming API (`decrypt_stream`) yields plaintext before the authentication tag is verified. Callers must use `decrypt_stream_strict` or opt in explicitly.

CAV-002 — Padding length-bucket leak. Ciphertext length reveals the power-of-two padding bucket. Callers needing full length-hiding must apply a fixed-frame transport.

CAV-003 — Ciphertext expansion. Noise injection produces 4–20× expansion, traded against traffic-analysis resistance.

12 Conclusion

We have presented NAPSEQ, an AEAD scheme whose entire security reduces to the pseudo-randomness of HMAC-SHA256. The construction avoids all algebraic structures targeted by known quantum algorithms, relies solely on FIPS-approved primitives, provides a conventional RFC 5116-compatible interface, and has been validated against a 30-vector KAT corpus, the full NIST SP 800-22 statistical suite, and a fuzz harness covering the complete decoder pipeline.

Known limitations include ciphertext expansion (inherent in the noise-injection design), a 16-bit plaintext length cap, and a padding length-bucket leak; these are documented and assigned to future wire-format versions.

Future work includes a formal security proof, a TVLA side-channel analysis of the Rust constant-time implementation, a FIPS 140-3 module boundary definition, and a performance evaluation on embedded ARM Cortex-M4 and RISC-V targets.

References

- [1] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, November 2007.
- [2] Y. Nir, A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 7539, May 2015.
- [3] C. Dobraunig, M. Eichlseder, F. Mendel, M. Schl  ffer, *Ascon v1.2: Lightweight Authenticated Encryption and Hashing*, Journal of Cryptology, 34(3), 2021.
- [4] L. K. Grover, *A fast quantum mechanical algorithm for database search*, Proceedings of STOC '96, pp. 212–219, 1996.
- [5] NIST, *Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process*, NIST IR 8413, July 2022.
- [6] A. Joux, *Authentication failures in NIST version of GCM*, NIST Comment, 2006.
- [7] T. Iwata, K. Ohashi, K. Minematsu, *Breaking and Repairing GCM Security Proofs*, CRYPTO 2012, LNCS 7417, pp. 31–49.
- [8] H. Krawczyk, P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, May 2010.
- [9] E. Barker, J. Kelsey, *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, NIST SP 800-90A Rev 1, June 2015.

- [10] M. Bellare, R. Canetti, H. Krawczyk, *Keying Hash Functions for Message Authentication*, CRYPTO 1996, LNCS 1109, pp. 1–15.
- [11] H. Krawczyk, M. Bellare, R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, February 1997.
- [12] A. Rukhin, J. Soto, J. Nechvatal, et al., *A Statistical Test Suite for Random and Pseudo-random Number Generators for Cryptographic Applications*, NIST SP 800-22 Rev 1a, April 2010.